

Snort RAG Rule Generator

Projet NLP - Sujet 2 : Generation de regles SNORT

Devoir 3 : Implementation et comparaison des architectures RAG

FST Tanger - Master NLP 2026

Champ	Valeur
Auteur / Groupe	A completer par les membres du groupe
Thematique	CyberSecurity
Sujet	Generation de regles SNORT
Date	Mai 2026

1. Resume executif

Ce livrable propose un systeme defensif base sur NLP et Retrieval-Augmented Generation (RAG) pour generer automatiquement des regles Snort a partir de descriptions d'attaques reseau. Le projet respecte le cahier de charge: dataset personnel, enrichissement synthetique, retrieval Top-k, generation controlee, classification du type d'attaque, visualisation/clustering, interface Gradio et comparaison des architectures du Devoir 3.

Comme le TP interdit l'utilisation d'API OpenAI, Mistral ou Ollama, le module de generation est local et transparent: il utilise les documents recuperes et des templates Snort valides au lieu d'appeler un LLM externe.

2. Contraintes du cahier de charge respectees

Contrainte	Implementation
Base personnelle	Dataset final genere dans data/processed a partir de scenarios manuels et variations synthetiques.
Sources reelles	Sources Snort/ET Open utilisees pour legitimer la syntaxe et les familles, sans copier un dataset public.
Pipeline RAG	Entree utilisateur, vectorisation, Top-k retrieval, prompt enrichi, generation de regle, explication.
Retrieval	TF-IDF, BM25 et fusion hybride; FAISS/sentence-transformers prevus en option.
Devoir 3	Baseline, RAG classique, re-ranking, hybride, multi-hop, graph RAG, agentic RAG, metriques, t-SNE, Gradio.
Interface	Dashboard Gradio avec choix d architecture et ajout de PDF comme nouvelle base de connaissance.

3. Sources reelles utilisees

Le projet n'integre pas directement un corpus public pre-etiquete comme dataset final. Les sources suivantes servent de references pour la syntaxe et la legitime des regles.

Source	URL	Usage
Snort 3 Rule Writing Guide - Cisco Talos	https://docs.snort.org/	Syntaxe: entetes, msg, sid, rev, classtype, content, http_uri, flow, detection_filter.
Snort.org downloads	https://www.snort.org/downloads	Page officielle pour versions Snort et regles communautaires/registered.
Registered vs Subscriber	https://snort.org/documents/registered-vs-subscriber	Contexte des jeux de regles: Community, Registered, Subscriber.
Emerging Threats Open Rules	https://rules.emergingthreats.net/open/snort-2.9.0/emerging.rules.tar.gz	Source ouverte optionnelle pour extraire des squelettes de regles et verifier les categories.
Emerging Threats FAQ	https://community.emergingthreats.net/t/frequently-asked-questions/56	Contexte ET Open et regles maintenues par la communaute.

4. Construction et enrichissement du dataset

Le dataset genere contient 216 lignes: 204 exemples malveillants et 12 exemples benins. Chaque ligne contient description naturelle, label, type d'attaque, famille, severite, regle Snort, statut de validation, source de reference et exemple de log.

La commande `python -m snort_rag.generate_dataset --multiplier N` permet d'augmenter automatiquement le dataset. Les variations incluent style SOC, requete courte, formulation technique, francais/anglais et exemples de logs.

Types inclus: benign, command_injection, directory_traversal, dns_axfr, dns_tunneling, ftp_bruteforce, icmp_sweep, log4shell, malware_c2, port_scan, rdp_bruteforce, shellshock, smb_exploit, sql_injection, ssh_bruteforce, suspicious_user_agent, webshell_upload, xss

5. Architecture logicielle

- src/snort_rag/rule_parser.py : parseur et validateur syntaxique Snort.
- src/snort_rag/templates.py : familles, mots-cles et templates Snort defensifs.
- src/snort_rag/retrieval.py : TF-IDF, BM25, retrieval hybride et ajout de PDF.
- src/snort_rag/generator.py : prompt, choix du type d attaque, generation locale et explication.
- src/snort_rag/architectures.py : implementation des 7 architectures RAG.
- src/snort_rag/evaluation.py : metriques, tests et visualisation t-SNE.
- src/snort_rag/app_gradio.py : interface web.

6. Pipeline RAG complet

1. L'utilisateur saisit une description d'attaque.
2. La requete est vectorisee; BM25 calcule un score lexical complementaire.
3. Le systeme recupere les documents Top-k similaires dans la base personnelle.
4. Un prompt enrichi est construit avec les regles et descriptions recuperees.
5. Le generateur local choisit le type d'attaque et remplit un template Snort adapte.
6. Le parseur verifie msg, sid, rev, classtype et les options utiles.
7. Le systeme retourne la regle, l'explication, le type d'attaque et les documents utilises.

7. Architectures comparees dans Devoir 3

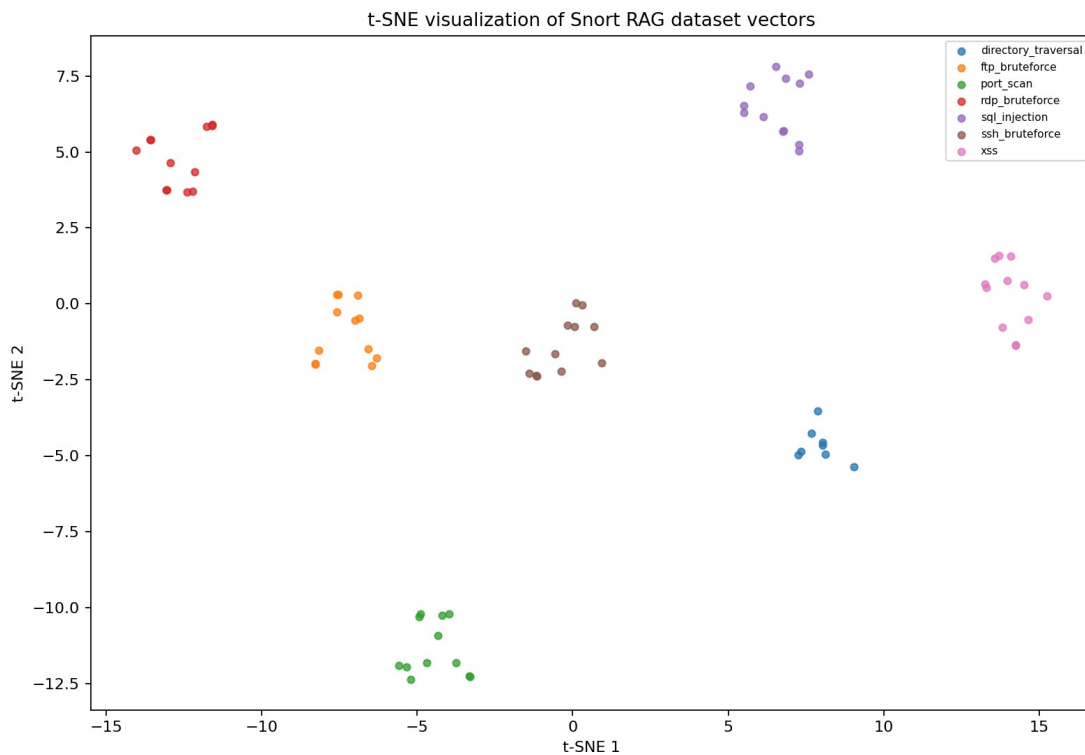
Architecture	Principe
Baseline	Generation directe depuis les mots-cles de la requete.
RAG classique	Retrieval sparse TF-IDF puis generation avec Top-k.
RAG + re-ranking	Re-classement selon type d attaque, validite et source.
RAG hybride	Fusion TF-IDF et BM25.
Multi-hop RAG	Retrieval, reformulation, deuxieme retrieval.
Graph RAG	Graphe type d attaque -> exemples/regles.
Agentic RAG	Choix automatique de strategie selon ambiguite.

8. Metriques et resultats

Metriques: accuracy de classification, precision/recall/F1 macro, taux de regles valides, retrieval hit@3, couverture moyenne des options Snort et risque d hallucination.

Architecture	F1	Validite	Hit@3	Risque halluc.
agentic_rag	1.000	0.917	1.000	0.021
graph_rag	1.000	0.917	1.000	0.033
multi_hop	1.000	0.917	1.000	0.033
rag_classic	1.000	0.917	1.000	0.033
rag_hybrid	1.000	0.917	1.000	0.033
rag_rerank	1.000	0.917	1.000	0.033
baseline	1.000	0.917	0.000	0.337

Visualisation des embeddings (t-SNE) produite par le script d evaluation:



9. Analyse critique

Les architectures RAG sont plus robustes que la baseline car elles recuperent un contexte documente avant generation. La baseline peut reconnaitre un type d'attaque simple, mais elle ne prouve pas l'origine de la reponse et garde le risque d'hallucination le plus eleve.

L'architecture la plus adaptee est Agentic RAG: elle choisit la strategie selon l'ambiguite, peut faire une expansion de requete et reduit le risque d'hallucination. Le RAG hybride est egalement solide car il combine similarite vectorielle et correspondance lexicale BM25.

Limites: les regles sont validees par un parseur interne, pas par un moteur Snort reel. Avant production, il faut tester avec Snort -T et sur PCAP. Les templates restent volontairement conservateurs pour reduire les faux positifs.

10. Mode d execution

```
pip install -r requirements.txt
pip install -e .
python -m snort_rag.generate_dataset --multiplier 20
python -m snort_rag.run_devoir3
python -m snort_rag.app_gradio
```

11. Conclusion

Le livrable fournit un socle complet pour le sujet Snort: dataset personnel enrichissable, pipeline RAG, generation defensivement controlee, interface web, evaluation des architectures et rapport. Il respecte l'objectif pedagogique: comprendre le retrieval et justifier les resultats au lieu d'utiliser un LLM comme boite noire.